sim6809

A headless Motorola 6809 simulator – user manual

plm09 project

October 2026

Contents

1 Introduction **2**

2 Building **2**

3 Command-line usage **2**

3.1 Example 3

4 Loading programs **3**

5 The machine **4**

5.1 Memory 4

5.2 Initial state 4

5.3 Memory-mapped I/O 4

6 The CPU **5**

6.1 Instruction set 5

6.2 Instructions that stop the simulator 5

7 Exit status **6**

8 Tracing **6**

9 Using sim6809 with plmc **7**

10 Verification **7**

11 Limitations **7**

12 Source code **8**

1 Introduction

`sim6809` is a command-line simulator for the Motorola MC6809 8-bit microprocessor. It loads a program in Motorola S-record format, runs it with no display or user interface, and reports the result through standard output and its exit status. This makes it a good fit for automated tests, scripts and continuous integration.

It was written for the plm09 project, a compiler that turns a subset of PL/M into 6809 assembly for the lwasm assembler. In that project it runs every compiled test program. Nothing in it depends on the compiler, though: it runs any 6809 program that can be expressed as S-records and does its I/O through the simulator's I/O block.

Main features

- The complete documented MC6809 instruction set, including all addressing modes (indexed, indirect and PC-relative) and the flag behavior given in the Motorola datasheet.
- A 16-byte memory-mapped I/O block for character output and input, printing a 16-bit decimal number, and ending the program with an exit status.
- An optional per-instruction trace with disassembly and register contents.
- An instruction limit, so a runaway program can't hang a test run.
- Distinct exit statuses for normal exit, illegal instruction and timeout.
- Memory filled with `$A5` at startup, so reads of uninitialized variables are easy to spot.

What it deliberately leaves out is covered in Limitations.

2 Building

`sim6809` is plain C11 with no dependencies beyond the C library. From the top of the plm09 tree:

```
make build/sim6809
```

`make sim-test` builds it and runs its self-tests (see Verification). Assembling the test programs needs lwasm from lwtools, and the instruction length test also needs python3.

3 Command-line usage

```
sim6809 [options] prog.s19
```

Option	Meaning
`-t`	Trace every instruction to standard error before executing it. See Tracing.
`-n N`	Stop after *N* instructions. The default is 100,000,000.
`--io BASE`	Put the I/O block at *BASE* instead of `$FF00`. *BASE* can be at most `$FFF0`.
`--entry ADDR`	Start execution at *ADDR*. Needed only if the file has no S9 record; if given, it overrides the S9 address.
`--zero`	Clear memory to `$00` instead of filling it with `$A5`.
`-h`, `--help`	Print a usage summary and exit with status 0.

Numbers given to options are **decimal** unless they start with `0x` or `$`. For example, `--entry 4096`, `--entry 0x1000` and `--entry '$1000'` all mean the same address, but `--entry 1000` means decimal 1000. Quote `$` values so the shell doesn't expand them.

Exactly one program file is accepted. Anything that isn't a recognized option is taken as the file name. A second file name, an unknown option or a bad number prints the usage summary and exits with status 2.

3.1 Example

```
; hello.asm -- print a string and a number, then exit with status 3
IO      EQU     $FF00
        ORG     $1000
START   LDS     #$8000
        LDX     #MSG
LOOP    LDA     ,X+
        BEQ     DONE
        STA     IO              ; character output
        BRA     LOOP
DONE    LDD     #1234
        STD     IO+4            ; decimal output
        LDB     #3
        STB     IO+2            ; exit with status 3
MSG     FCC     "HELLO"
        FCB     10,0
        END     START
```

```
$ lwasm -9 -f srec -o hello.s19 hello.asm
$ sim6809 hello.s19
HELLO
1234
$ echo $?
3
```

`lwasm -9` keeps lwasm in 6809 mode (by default it accepts 6309 instructions), and `END START` makes lwasm write the S9 start-address record that `sim6809` uses as its entry point.

4 Loading programs

`sim6809` reads Motorola S-records:

Record	Handling
`S0` (header)	ignored
`S1` (data, 16-bit address)	bytes stored in memory at the given address
`S5` (record count)	ignored
`S9` (end, 16-bit address)	sets the start address
`S2`, `S3`, `S7`, `S8` (24/32-bit)	error

Every record's checksum is verified. A malformed line, a bad hex digit or a checksum mismatch stops loading with a message giving the line number, and the simulator exits with status 2. Blank lines and both LF and CR LF line endings are accepted. Data wraps around at `$FFFF`.

If a file contains more than one S9 record, the last one wins. That makes it easy to combine programs: `cat prog.s19 monitor.s19 > both.s19` loads both, and execution starts at `monitor.s19`'s entry point. The plm09 test suite uses this to load a stand-in Color BASIC ROM together with a compiled program.

lwasm's `RMB` reserves memory without writing any bytes, so uninitialized variables don't make the S-record file larger. They start with the fill value (`$A5`, or `$00` with `--zero`).

5 The machine

5.1 Memory

The 64 KB address space is all RAM, except for the 16-byte I/O block. There's no ROM and no memory protection: programs can write anywhere, including over their own code and the interrupt vectors at `$FFF0`–`$FFFF`. Nothing reads the vectors, because the simulator starts at the entry address and has no interrupts.

5.2 Initial state

Register	Value
PC	the S9 address, or `--entry`
A, B, X, Y, U, DP	0
S	**0**
CC	`$50` (I and F set, as after a hardware reset)

S starts at zero, so **a program must load S before pushing anything.** Otherwise the first push wraps around to `$FFFF`/`$FFFE`. (That is ordinary RAM in the simulator, so it doesn't crash, but it isn't what you want.)

5.3 Memory-mapped I/O

The I/O block is 16 bytes long, at `$FF00` by default (see `--io`). Offsets are relative to its base:

Offset	Access	Function
+0	write	Write the byte to standard output.
+1	read	Read one byte from standard input. Returns `$00` at end of input.
+2	write	Stop, with the byte as the simulator's exit status.
+4	write	Store the high byte for +5.
+5	write	Print (high byte from +4) × 256 + this byte as an unsigned decimal number, followed by a newline.

Other offsets, and reads of the write-only offsets, return `$00`. Writes to unused offsets are ignored.

- **Printing numbers:** `STD $FF04` writes A to +4 and then B to +5, so it prints D in one instruction. A single write to +5 prints using whatever +4 last held.
- **Exiting:** the exit write takes effect when the current instruction finishes. Only the first one counts. Standard output is flushed before the simulator exits.
- **Input:** stdin is read through the C library, so input can come from a file or a pipe (`sim6809 prog.s19 < input.txt`). With no input, every read returns 0, just as at end of input.
- **Instruction fetches** always read memory directly and never touch the I/O block, so code that runs into the I/O addresses never consumes input. Every other access goes through the I/O block, including stack pushes and pulls, the bytes of 16-bit operands, and pointers read for indirect addressing.

6 The CPU

6.1 Instruction set

The simulator implements all documented MC6809 instructions, on all three opcode pages (`$10` and `$11` prefixes), with every addressing mode: inherent, immediate, direct (using DP), extended, relative (8- and 16-bit), and indexed.

The indexed forms are:

- constant offsets of 5, 8 and 16 bits from X, Y, U or S;
- accumulator offsets `A,R`, `B,R`, `D,R` (sign-extended);
- auto-increment and auto-decrement `,R+`, `,R++`, `,-R`, `,--R`;
- PC-relative with 8- and 16-bit offsets;
- indirect versions of every form that has one, plus extended indirect `[addr]`.

The condition codes follow the Motorola datasheet. Where the datasheet leaves a result undefined, the simulator behaves as follows:

Case	Behavior
H after subtract and compare	unchanged
V after `DAA`	cleared
V after `SEX`	unchanged
`TFR`/`EXG` from an 8-bit to a 16-bit register	high byte is `$FF` (as on the real MC6809)
`TFR`/`EXG` from a 16-bit to an 8-bit register	low byte is used
`TFR`/`EXG` naming an invalid register code	reads `$FF`/`$FFFF`, writes are ignored

`RTI` honors the E flag: it pulls the whole register set when E is set, and only CC and PC when it is clear.

6.2 Instructions that stop the simulator

The simulator has no interrupt sources, so instructions that only make sense with interrupts stop it, along with anything that isn't a valid MC6809 instruction:

Instruction	Message
`SWI`, `SWI2`, `SWI3`	`SWI`, `SWI2`, `SWI3`
`SYNC`, `CWAI`	`SYNC`, `CWAI`
undocumented opcodes (e.g. `$01`, `$02`, `$10 $00`)	`illegal opcode`
`$87`, `$8F`, `$C7`, `$CD`, `$CF`, `$10 $8F`, `$10 $CF`	`illegal opcode (store immediate)`
reserved indexed postbytes (`[,R+]`, `[,-R]`, modes 7, A and E, or `$8F`/`$AF`/… with no indirection)	`illegal indexed mode` / `illegal indexed postbyte`

In each case the simulator prints the message, the address of the offending instruction and all registers to standard error, then exits with status 125:

```
sim6809: illegal opcode at PC=1001
  A=00 B=00 X=0000 Y=0000 U=0000 S=0000 DP=00 CC=-F-I----
```

The undocumented MC6809 opcodes (for example `$01` as a NEG alias, or `$3E` RESET) are treated as illegal rather than emulated. Code that relies on them is almost always a bug.

7 Exit status

Status	Meaning
0–255	The program wrote this value to the exit port (+2).
124	The instruction limit (`-n`) was reached. A message on stderr gives the PC.
125	Illegal opcode or postbyte, or `SWI`/`SWI2`/`SWI3`/`SYNC`/`CWAI`. A register dump goes to stderr.
2	Usage error, unreadable file, bad S-record, or no start address.

A program can itself exit with 124, 125 or 2, so a script that needs to tell these cases apart should also check standard error. (124 is also the status the `timeout` command uses when it stops a command.)

A program that never writes the exit port runs until the instruction limit:

```
$ sim6809 -n 5 hello.s19
Hsim6809: instruction limit (5) reached at PC=100E
$ echo $?
124
```

8 Tracing

With `-t`, the simulator prints one line to standard error **before** each instruction executes:

```
1000 10CE8000   LDS   #$8000         A=00 B=00 X=0000 Y=0000 U=0000 S=0000 DP=00 CC=-F-I----
1004 8E101B     LDX   #$101B         A=00 B=00 X=0000 Y=0000 U=0000 S=8000 DP=00 CC=-F-IN---
1007 A680       LDA   ,X+            A=00 B=00 X=101B Y=0000 U=0000 S=8000 DP=00 CC=-F-I----
1009 2705       BEQ   $1010          A=48 B=00 X=101C Y=0000 U=0000 S=8000 DP=00 CC=-F-I----
100B B7FF00     STA   $FF00          A=48 B=00 X=101C Y=0000 U=0000 S=8000 DP=00 CC=-F-I----
100E 20F7       BRA   $1007          A=48 B=00 X=101C Y=0000 U=0000 S=8000 DP=00 CC=-F-I----
```

The columns are:

1. **Address** of the instruction.
2. **Instruction bytes** in hex: opcode, any prefix, postbyte and operands, up to 5 bytes.
3. **Disassembly** in lwasm-compatible syntax.
   - Direct-page operands are shown as `<$nn`.
   - Branch targets and PC-relative operands are shown as the absolute address they refer to.
   - Undecodable bytes are shown as `???`.
4. **Registers** as they are *before* the instruction runs.
   - CC is spelled out as `EFHINZVC`, with `-` for a clear flag.

Because the trace goes to standard error and program output to standard output, you can separate them:

```
sim6809 -t prog.s19 > output.txt 2> trace.txt
```

Programs often run millions of instructions, so a trace can get large. A lower `-n` limit, `head`, or `grep` on an address range keeps it manageable. For example, to see what a routine at `$1234` was called with:

```
sim6809 -t prog.s19 2>&1 >/dev/null | grep '^1234 '
```

9 Using sim6809 with plmc

plmc's simulator hooks (`runtime/sim/plm_hooks.asm`) implement the PL/M `EXTERNAL` procedures `PUTCH`, `GETCH`, `PUTDEC` and `EXIT` with the I/O block above, and put the stack top at `$FF00`. The whole cycle is:

```
build/plmc -o prog.asm prog.plm
lwasm -9 -f srec -I runtime -I runtime/sim -o prog.s19 prog.asm
build/sim6809 prog.s19 < input.txt
```

`make run F=prog.plm` does the same in one step. When a program misbehaves, the generated `.asm` has each PL/M statement as a comment (`; 12: X = X + 1;`), and the trace addresses can be matched against lwasm's listing (`--list=prog.lst`).

10 Verification

The simulator is checked by `make sim-test`, which runs:

- **Hand-checked tests.**
  - *ALU:* the 8- and 16-bit arithmetic and logic instructions, checked result and flags against tables worked out by hand from the datasheet, with boundary values such as `$7F`, `$80`, `$FF` and `$00`.
  - *Branches:* all 16 short and 16 long conditional branches, each tried with all 16 combinations of N, Z, V and C.
  - *Indexed addressing:* every indexed form, including indirect and PC-relative, plus direct addressing with a non-zero DP.
  - *Stack and transfers:* `PSHS`/`PULS`/`PSHU`/`PULU` ordering, `JSR`, `BSR`, `LBSR`, `RTS`, `PULS PC`, `RTI` with E clear and set, and `TFR`/`EXG`.
  - *Edge cases:* an illegal opcode and the instruction limit, which must give exit statuses 125 and 124.
- **Instruction-length check against lwasm.** A generated program contains every opcode in every addressing mode, about 11,000 instructions, laid out so execution falls straight through. The program is run with `-t`. Every traced address and its instruction bytes must match the address and code columns of lwasm's listing. This checks the instruction decoder in the CPU and the one in the disassembler independently, against the assembler.
- **Mutation checks.** During development, bugs were introduced on purpose (wrong flags, a 16-bit offset decoded as 8-bit) to confirm that the tests catch them.

11 Limitations

- **No interrupts.** There are no IRQ, FIRQ or NMI sources, and the instructions that wait for or raise interrupts stop the simulator.
- **No timing.** Instructions take no cycles, so timing loops run instantly and cycle counts aren't reported.
- **No 6309.** The Hitachi 6309's native mode and extra instructions aren't supported; they stop the simulator as illegal opcodes.
- **No real hardware.** There is no ROM, no video or other machine-specific hardware, no bank switching, and no memory beyond 64 KB. For machine-accurate emulation of a Color Computer or Dragon, use XRoar or MAME.
- **One input format.** Only S1/S9 S-records are loaded. Convert other formats with lwasm or a tool such as `srec_cat`.
- **Memory is not saved.** The memory image can't be inspected or saved after the run; print values through the I/O block instead.

12 Source code

File	Contents
`sim/cpu6809.c`, `cpu6809.h`	CPU core: registers, flags, addressing modes, instruction execution, I/O block
`sim/disasm.c`	table-driven disassembler used by the trace
`sim/srec.c`, `srec.h`	S-record loader
`sim/main.c`	command-line handling, run loop, trace and diagnostics
`sim/tests/`	self-test programs and expected output; `harness.inc` provides the check macros, `gen_len_ops.py` generates the length test

The core can be embedded in another program:

- `cpu_init(&cpu, io_base, fill)` resets the CPU and memory.
- Set `cpu.pc` and fill `cpu.mem`.
- Call `cpu_step(&cpu)` until it returns a non-zero `STOP_*` reason.
- `disasm(mem, pc, buf, len)` formats one instruction and returns its length.